

Bridging LLMs and Database Systems: A Deep Dive into Enhanced Relational Operators

Tianjing Zeng, Yin Lin, Rong Zhu, Yunxiang Su,

Zhongjun Ding, Bolin Ding, Jingren Zhou

 **Alibaba**

Foundation: Relational Algebra Theory

Every SQL query compiles into a **small, closed set of algebraic operators** — Edgar F. Codd, CACM 13(6), 1970

Algebra	Meaning	SQL
$\sigma_p(R)$	Selection	WHERE p
$\pi_a(R)$	Projection	SELECT A
$R \bowtie_{\theta} S$	θ -Join = $\times + \sigma$	JOIN ... ON θ
$\gamma_{a:e}(R)$	Grouping / aggregation	GROUP BY ... COUNT, SUM
$U - \times p$	Set ops & rename	UNION / EXCEPT / ...

“Departments with more than 5 employees earning over 3000”



```
SELECT dept FROM Employee
WHERE salary > 3000
GROUP BY dept HAVING COUNT(*) > 5;
```



$\sigma_{cnt>5}(\gamma_{dept; COUNT(*) \rightarrow cnt}(\sigma_{salary>3000}(Employee)))$

Same query, three views — the algebra is what the optimizer rewrites.

The power: a closed algebra — composable, rewritable, optimizable.

From One Algebra to Modern Analytics

Relational algebra evolved into the foundation of today's analytics stacks — for data analysis and for ML data preparation.

Relational Algebra

$\sigma \cdot \pi \cdot \bowtie \cdot \gamma$



fifty years of engines:

OLTP row stores · warehouses · distributed & cloud SQL · streaming · embedded

... and gradually became the foundation of ↓

SQL analytics (OLAP) — *for data analysis*

scans · joins · group-by over billions of rows

"revenue by segment" · "top product per region"



Data science pipelines — *for data science & ML prep*

EDA · cleaning · feature engineering · labeling

`df.merge(...)` = $\bowtie$ `df.groupby(...)` = γ

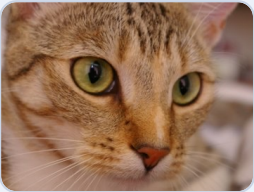
+ beyond RA: `df.apply(f)` UDFs · `pivot / melt` · row order



Warehouse or notebook, the contract is the same: **exact predicates over structured data.**

The Data Changed Too: Multi-Modal Tables

Tables now carry **photos, free text, and documents** next to numbers — and analytics wants to query their content.

ID	Name	Photo	Customer review	Manual
101	Coffee Mug Set		"arrived chipped — very disappointed"	Care Guide PDF
102	Fruit Basket		"fresh and beautifully packed"	Storage Tips PDF
103	Cat Plush Toy		"my cat ignores it completely"	Safety Sheet PDF

"Which product photos show food?"

→ a predicate over *PIXELS*

"Which reviews complain about damage?"

→ a predicate over *PROSE*

"Which manuals warn about choking hazards?"

→ a predicate over *DOCUMENTS*

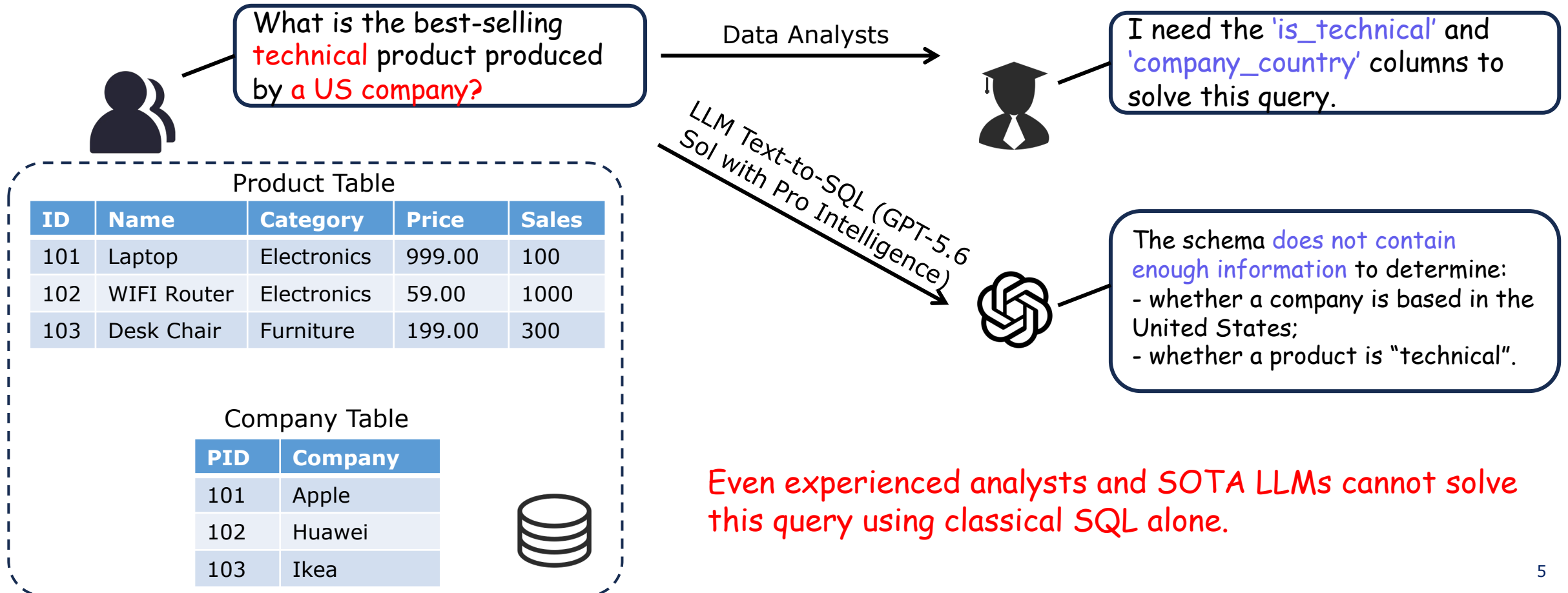
Classical SQL cannot reach the meaning:

```
WHERE photo = ? -- meaningless
WHERE review LIKE '%damage%' -- 0 rows
WHERE manual LIKE '%choking%' -- binary PDF: can't even read it
```

The values are stored — their **meaning** is not. No exact predicate reaches it.

Databases Are Powerful, but Query-Bounded

Modern data analytics raises new demands: users increasingly ask questions beyond classical SQL — and beyond the database storage.



Databases Are Powerful, but Query-Bounded

Classical SQL cannot solve real-world queries that **demand semantic reasoning** at the tuple level or **go beyond database storage** (requiring world knowledge)



What is the best-selling **technical** product produced by **a US company**?

Product Table

ID	Name	Category	Price	Sales
101	Laptop	Electronics	999.00	100
102	WIFI Router	Electronics	59.00	1000
103	Desk Chair	Furniture	199.00	300

Company Table

PID	Company
101	Apple
102	Huawei
103	Ikea



① whether a product is “technical”?

⇒ Requires tuple-level semantic reasoning based on the product name.

② whether a company is based in the United States?

⇒ Requires world knowledge to infer the location of each company.

Neither gap is new — the DB community has been tackling both for two decades:

① semantic matching → **fuzzy join**; ② world knowledge → **beyond-database queries**

Gap ① in the Pre-LLM Era: Fuzzy Join

We need joins on **semantic equivalence**: $R \bowtie_{\text{match}(r, s)} S$, not string equality.

Customer Id	Company Name		Sale Id	Client Name	Revenue
C101	International Business Machines	↔	S001	IBM	\$500,000
C102	Microsoft Corporation	↔	S002	Microsoft Corp.	\$1,200,000
C103	Google LLC	↔	S003	Google Inc.	\$700,000
C104	Procter & Gamble Co.	↔	S004	P&G	\$1,000,000
C105	Deloitte Consulting LLP	✗	S005	L.E.K. Consulting	\$450,000

"P&G" ↔ "Procter & Gamble": same company, no shared key → equi-join returns nothing.

Why string similarity is not enough:

$\text{sim}(\text{"P\&G"}, \text{"Procter \& Gamble"}) = 0.08 \rightarrow$ **missed match ✗**

$\text{sim}(\text{"Apple Inc."}, \text{"Apple Bank"}) = 0.85 \rightarrow$ **false match ✗**

① Similarity join (string-level)

Edit distance, Jaccard / q-grams, TF-IDF cosine with a threshold τ ; scaled by prefix-filtering algorithms (SSJoin [1], PPJoin [2]).

② Semantic join via table corpora

SEMA-JOIN [3]: mines co-occurrence statistics from 100M+ web tables to join related values (country name $\bowtie$ country code) with zero string overlap.

Limitations: similarity functions and thresholds are hand-tuned per domain; corpus statistics fail on private, long-tail, or fresh data; no way to express matching intent ("same company" vs. "same parent company").

Gap ② in the Pre-LLM Era: Beyond-Database Queries

SQL runs under the **closed-world assumption**: it cannot answer questions whose facts are not stored in any column [1].

"Among these MVPs, who was drafted earliest?"



Player Name	Current Team	First NBA MVP Award	Draft Year
Nikola Jokić	Nuggets	2021	2014
Giannis Antetokounmpo	Bucks	2019	2013
Stephen Curry	Warriors	2015	2009
Russell Westbrook	Clippers	2017	2008
Joel Embiid	76ers	2023	2014

"Draft Year" (orange) is not a stored column — the answer lives outside the database.

① Manual ETL & federation

Analysts find an external source, wrangle it, and load new columns; mediators query remote structured sources.

② Web tables & knowledge bases

Auto-fill columns from web-table corpora (WebTables [2], InfoGather [3]) or curated KBs (Freebase, YAGO).

③ Crowd-powered databases

CrowdDB [4], Qurk, Deco: ask human workers for missing values at query time.

Limitations: per-query human effort; coverage limited to structured, curated sources (stale & incomplete); crowds are slow and costly.

Why Are LLMs a Natural Fit for Both Gaps?

Web-scale pretraining gives one model both **semantic understanding** and **world knowledge** — exposed through a single natural-language interface.

 OpenAI  Claude  Gemini  Grok  Qwen  deepseek

Fuzzy matching → semantic reasoning

“Do ‘P&G’ and ‘Procter & Gamble’ refer to the same company?”
→ “Yes.”

Judges equivalence directly — **no similarity function, no threshold τ**

Robust to abbreviations, synonyms, typos, and cross-lingual names

Matching intent is programmable in plain language (“same parent company”)

Missing data → parametric world knowledge

“What year was Stephen Curry drafted?”
→ “2009.”

Pretraining corpus acts as a **vast implicit table** — no manual ETL, no curated KB required

Answers open-ended, tuple-level questions on demand

Pairs with retrieval / web search for freshness and verification

⇒ **Wrap LLM calls inside relational operators** (`llm_join`, `llm_filter`, `llm_impute`, ...) — one uniform, declarative API replaces bespoke per-task pipelines.

But LLM calls are **costly and fallible** — exactly the operator- and system-level questions this tutorial addresses.

Why an Operator, Not a Raw API Call?

A semantic operator puts the LLM call **inside the relational operator**.

✗ One giant chat

“Here are 1,000,000 rows – which are technical?”

Doesn't fit. Unparseable. Unauditable.

✗ Hand-written glue code

```
for row in table: api.chat(my_prompt(row))
```

Re-invented per user. Invisible to the optimizer.

✓ Fixed prompt template, inherit the semantics of relational operators

```
llm_filter(product, 'is a technical product') → “Is {name} technical? yes / no”
```

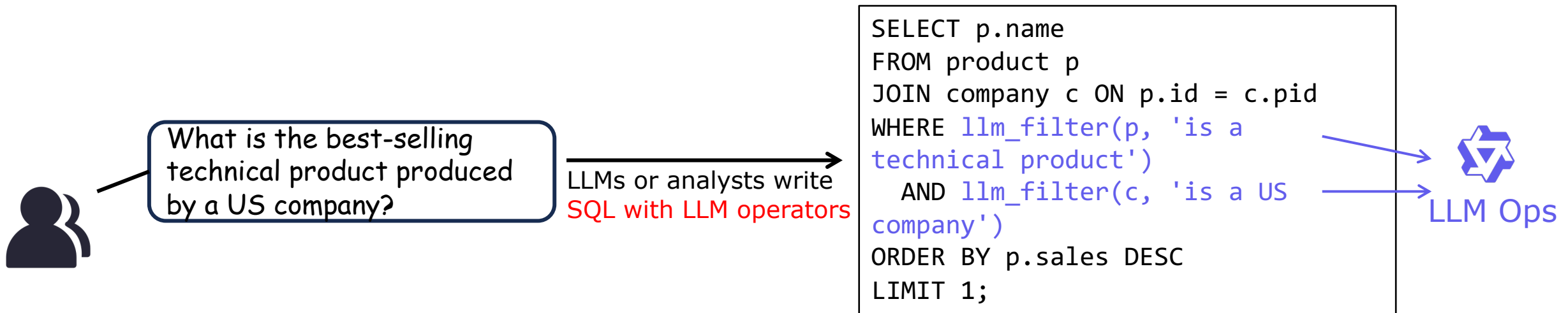
LLM = σ 's predicate · π 's function · $\bowtie$'s matcher · τ 's comparator · γ 's grouper

Declarative $\Rightarrow$ batch · cache · reorder · cascade

LLM Operators Can Extend SQL Capabilities

LLMs are **powerful semantic reasoners** with **extensive world knowledge**, motivating the development of LLM operators and their integration into SQL.

A brief definition: LLM-enhanced Relation Operators (**LROs**) invoke LLMs during operator execution to enable world-knowledge retrieval and semantic reasoning.



How LLM Operators Execute: llm_filter

```
llm_filter(product, 'is a technical product')
```

Product Table

ID	Name	Category	Price	Sales
101	Laptop	Electronics	999.00	100
102	WIFI Router	Electronics	59.00	1000
103	Desk Chair	Furniture	199.00	300

Tuple-wise
prompting



...



...

Is {laptop} a technical product? **Explain your reasoning**, and end your response with either "yes" or "no."

↓ Filter tuples with a "yes" answer

ID	Name	Category	Price	Sales
101	Laptop	Electronics	999.00	100

$$\sigma_p(R) \rightarrow \sigma_{\text{LLM}(\mathbf{t}, \mathbf{r})}(R)$$

$\mathbf{t}$ = "Is {name} a technical product?"

predicate: exact comparison $\rightarrow$ NL condition, per tuple

More LLM Operators: LLM-Enhanced Join

Beyond filtering, many other SQL operators can be enhanced with LLMs.

LLM-Enhanced Join: Join customer information (left table) with sales revenue (right table) by identifying records that refer to the same company.

CustomerID	CompanyName		SaleID	ClientName	Revenue
C101	International Business Machines	↔	S001	IBM	500,000
C102	Microsoft Corporation	↔	S002	Microsoft Corp.	1,200,000
C103	Google LLC	↔	S003	Google Inc.	700,000
C104	Proctor & Gamble	↔	S004	P&G	1,000,000
C105	Deloitte Consulting LLP	✗	S005	Reckitt	450,000

$R \bowtie_{\theta} S \rightarrow R \bowtie_{\text{LLM}(t, r, s)} S$
 $t = \text{"Do \{r\} and \{s\} name the same company?"}$
condition: key equality $\rightarrow$ semantic equivalence, on pairs

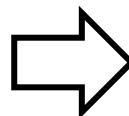


LLMs perform fuzzy matching — closing Gap ①.

More LLM Operators: LLM-Enhanced OrderBy

LLM-Enhanced OrderBy: Sort customer reviews of a pizzeria from most positive to most negative.

ReviewID	CustomerReview
0000001	The worst pizza I've ever had—bland, soggy, and overpriced.
0000002	Great atmosphere, but the pizza is only average.
0000003	The pizza was absolutely divine! Best carbonara I've ever had.
0000004	Good pizza with a crispy crust and fresh, flavorful toppings.
0000005	Pizza was okay, but the tiramisu was incredible and rich.



ReviewID	CustomerReview
0000003	The pizza was absolutely divine! Best carbonara I've ever had.
0000004	Good pizza with a crispy crust and fresh, flavorful toppings.
0000005	Pizza was okay, but the tiramisu was incredible and rich.
0000002	Great atmosphere, but the pizza is only average.
0000001	The worst pizza I've ever had—bland, soggy, and overpriced.

$T_A(R) \rightarrow T_{LLM(t, r, r')}(R)$
 t = “Which review is more positive?”
sort key: *stored column* $\rightarrow$ *pairwise NL comparison*

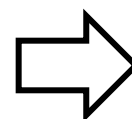


LLMs reason over review content and compare reviews relatively.

More LLM Operators: LLM-Enhanced GroupBy

LLM-Enhanced GroupBy: Group companies by type—technology or retail—and calculate the total revenue for each group.

ID	CompanyName	Revenue
S001	IBM	500,000
S002	Microsoft Corp.	1,200,000
S003	Google Inc.	700,000
S004	P&G	1,000,000
S005	Reckitt	450,000



GroupType	SUM(Revenue)
Technology	2,400,000
Retail	1,450,000

$\mathbf{Y}_{A; \text{agg}}(\mathbf{R}) \rightarrow \mathbf{Y}_{\text{LLM}(\mathbf{t}, \mathbf{r}); \text{agg}}(\mathbf{R})$
 $\mathbf{t}$ = “Is {company} tech or retail?”
grouping key: generated label;
aggregation stays classical



LLMs reason about company characteristics
and group companies by type.

LLM Alone, SQL Alone, or Hybrid?

Three ways to answer our query: an LLM alone, a SQL engine alone, or a **hybrid** — **LLM calls inside relational operators**.

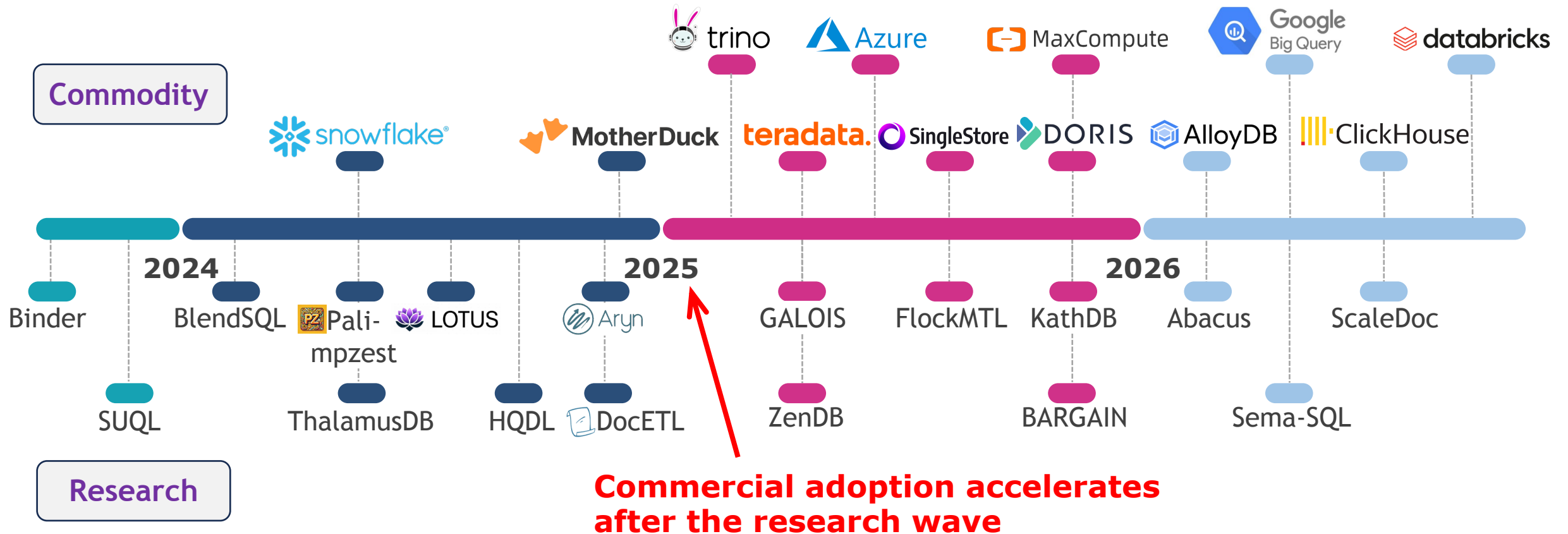
	LLM alone whole table in the prompt	SQL engine alone plain SQL	Hybrid: semantic operators LLM calls inside the engine
Semantics	✓ understands meaning & world knowledge	✗ exact values only — no semantics	✓ LLM handles the semantic predicates
Computation	✗ arithmetic in tokens — errors compound as data grows	✓ exact — scan, join, aggregate	✓ engine does all the math, exactly
Cost	minutes of tokens for a few KB of data	milliseconds, any size	LLM calls only where semantics is needed
Scales?	✗ accuracy drops with input size (context rot)	✓ scales — but can't express the query	✓ best of both — declarative & optimizable

Semantics → model. Computation → engine.

The semantic operator is that hand-off — systematic, declarative, optimizable.

An Emerging Trend of LLM Operators

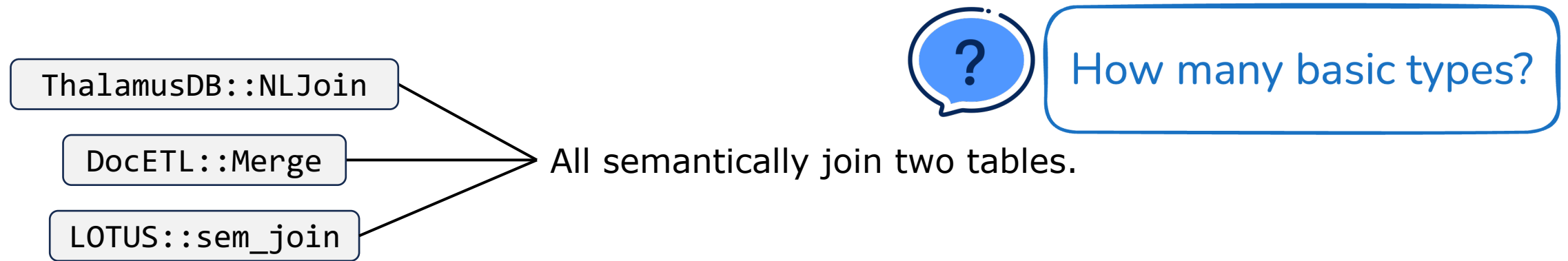
Numerous commodity databases and researches design LLM operators.



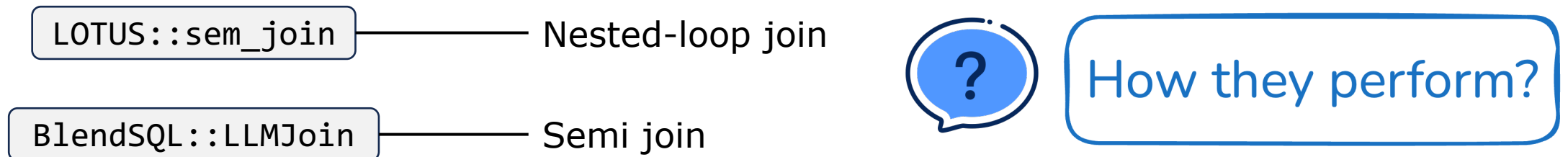
The debut dates are approximate; please refer only to each system's name and year.

Challenges at the Operator Level (Q1–Q2)

1. Despite using different names, operators across different systems share the same underlying operating intent. *How many distinct operating intents underlie current LROs?*

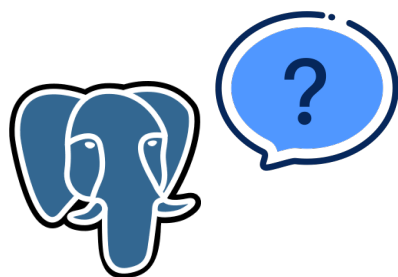


2. LROs with the same name, intent and operand may be implemented under various strategies across systems. *How do different LRO implementations impact their performance?*



Challenges at the System Level (Q3–Q4)

3. Real-world applications rely on suites of LROs rather than standalone operators. *What are the key concerns of the system design?*



With so many operators and implementations available, how should we navigate the design space?

LLM filter

LLM group

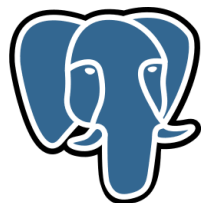
LLM join

LLM impute

LLM order

... ..

4. Beyond their initial design, *What are the key directions for optimizing multi-LRO systems?*



LRO Library

LLM filter

LLM join

LLM order

LLM group

LLM impute



How to optimize the efficiency and the cost?
Predicate pushdown? Model cascade? ...

Tutorial Roadmap

Five sections organized around **four questions**, moving from individual operators to complete systems.

1 **Foundation** — What is an LRO? How does it work? Why does it matter? (this introduction)

2 **Taxonomy of LROs** — Existing systems and the basic operating intents behind their operators.

3 **Benchmarks & Evaluation** — Existing benchmarks, their design, and head-to-head evaluation results.

4 **From Operators to Systems** — Multi-LRO design space, query planning, and our best practice.

5 **System Optimization** — Predicate pushdown, reordering, smart batching, model cascades, cost-based optimization.

Q1: *How many distinct operating intents underlie current LROs?*

Q2: *How do different LRO implementations impact their performance?*

Q3: *How should multi-LRO systems be designed?*

Q4: *What are the key directions for optimizing multi-LRO systems?*

Foundation: Defining LROs

What LROs Are: A Formal Definition

- An LLM-enhanced relational operator (LRO) is a logical extension of SQL that enables operations beyond the capability of traditional SQL. Formally, an LRO takes as input one or multiple **relations** $\mathcal{R}$, an **operand granularity** $g \in \{cell, row, column, table\}$, and a **natural-language requirement** l , and produces an output relation R' :

$$\text{LRO}(\mathcal{R}, g, l) \rightarrow R',$$

- where the LRO treats the input relation(s) as cells, rows, columns, or tables according to g , **invokes an LLM** to transform the input according to l , and materializes the result as a new relation R' .

What LROs Are: A Brief Definition

- Database operators that
 - operate over relational data
 - take a natural-language requirement
 - invoke LLMs during operator execution
 - produce relational outputs

An example table of Restaurants from Yelp

Name	Location	Description
Alley Wok	10 Market St, Long Beach, CA	A cozy home-style Chinese eatery with light seasoning.
Harbor Bar	Oakland, California, 200 E	A bar serving local catches and simple preparations.
Casa Oliva	45 Hanover St -- Boston	An Italian eatery serving classic comfort dishes.

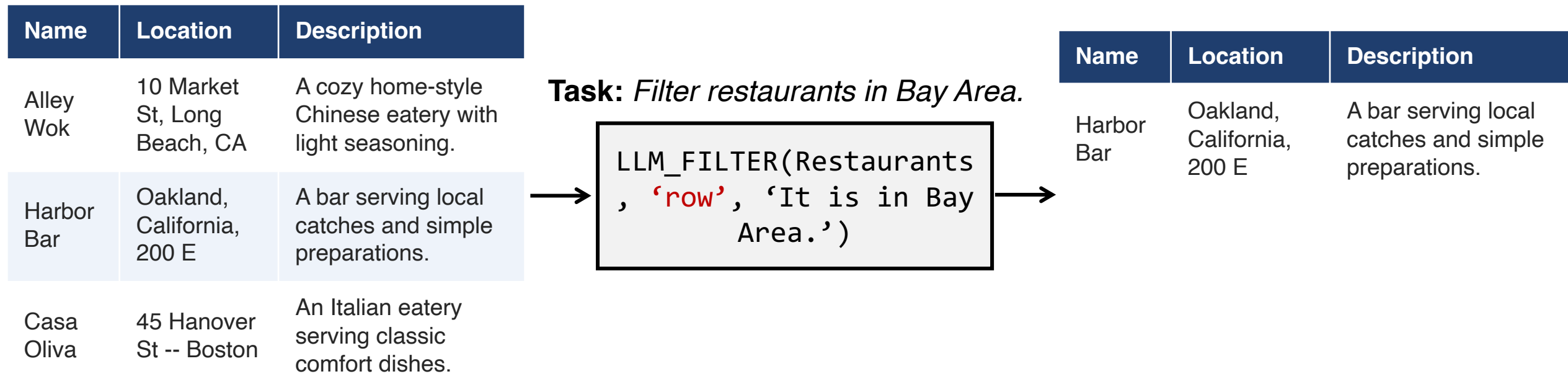
Task: *Filter restaurants in Bay Area.*

```
SELECT * FROM restaurants
WHERE LLM_FILTER(restaurants,
'row', 'It is in Bay Area.')
```

LROs are often integrated into SQL-based database systems or DataFrame frameworks such as pandas.

What LROs Are: An Example

- An LRO LLM_FILTER takes as input a relation with restaurant records, performs semantic filtering, and outputs a filtered relation with “Alley Wok” and “Harbor Bar”.
- Note that traditional SQL falls short because “which restaurant/row is in Bay Area” requires **world knowledge**.

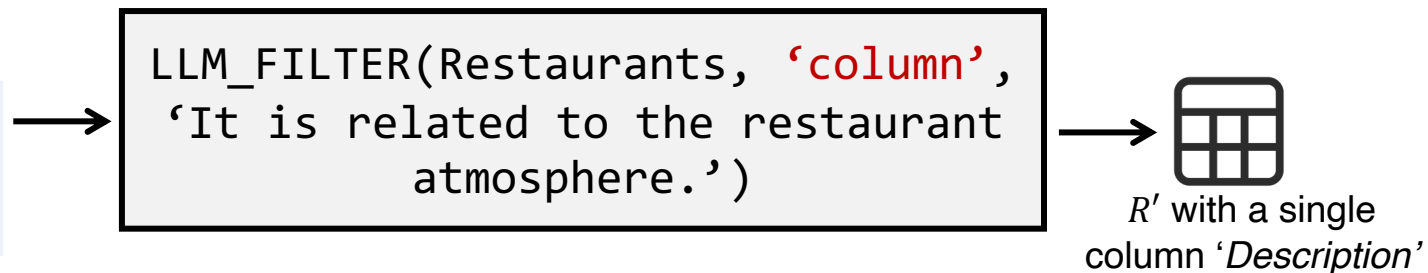


Operand Granularities Matter

- The LRO LLM_FILTER can also perform filtering **on columns**, that is, semantic schema linking, and outputs a relation whose schema is narrowed to include only the “Description” column.
- Note that traditional SQL falls short because “which column is related to atmosphere” requires **semantic reasoning**.

Name	Location	Description
Alley Wok	10 Market St, Long Beach, CA	A cozy home-style Chinese eatery with light seasoning.
Harbor Bar	Oakland, California, 200 E	A bar serving local catches and simple preparations.
Casa Oliva	45 Hanover St -- Boston	An Italian eatery serving classic comfort dishes.

Task: *Identify and select the columns relevant to the restaurant atmosphere.*



Operand Granularities Matter

These two LROs target different operand granularities, but have the same processing logic: to filter a subset.

```
LLM_FILTER(Restaurants, 'row',  
           'It is in Bay Area.')
```

Row-level LLM Filtering

```
LLM_FILTER(Restaurants, 'column',  
           'It is related to the restaurant  
           atmosphere.')
```

Column-level LLM Projection

Subset
filtering

Operand granularity should be treated as a dimension orthogonal to the operator's processing logic. So, we mention the operand granularity in LRO definitions.

What LROs Are Not

■ LROs ≠ Text-to-SQL

- Text-to-SQL uses LLMs to translate user intent into SQL without modifying the underlying operators, whereas LROs invoke LLMs during operator execution.

■ LROs ≠ embedding-based RAG

- Embedding-based RAG relies on vector similarity during retrieval, rather than LLM-based semantic manipulation.

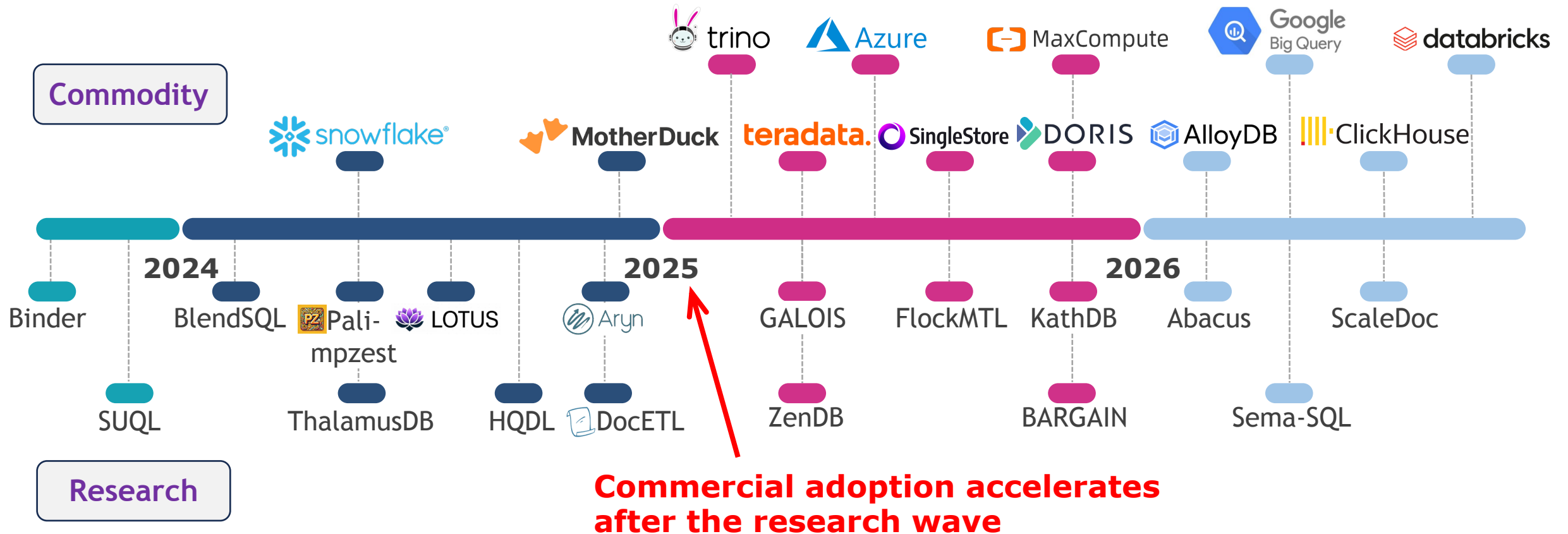
■ LROs ≠ General NLP tasks

- We exclude general NLP tasks that consume relational data but produce unstructured outputs, such as answers or summaries.

Current States & Taxonomy

An Emerging Trend of LLM Operators

Numerous commodity databases and researches design LLM operators.



The debut dates are approximate; please refer only to each system's name and year.



Snowflake Cortex

Cortex AI SQL functions for governed, high-throughput AI processing and analytics over enterprise structured and unstructured data.

AI FUNCTION SURFACE

STRUCTURED PREDICATES

AI_FILTER · AI_CLASSIFY

AGGREGATION

AI_AGG · AI_SUMMARIZE_AGG

GENERATION & EXTRACTION

AI_COMPLETE · AI_EXTRACT · AI_TRANSLATE

DOCUMENTS & VECTORS

AI_PARSE_DOCUMENT · AI_TRANSCRIBE · AI_EMBED

Warehouse-native, governed SQL surface — but not every function is an LRO.



Google BigQuery

SQL-native semantic operators for filtering, joining, scoring, and aggregating structured or multimodal data at warehouse scale.

AI FUNCTION SURFACE

PREDICATES

AI.IF · AI.CLASSIFY

RANKING & AGGREGATION

AI.SCORE · AI.AGG

GENERATION

AI.GENERATE · AI.GENERATE_TABLE

VECTORIZATION

AI.EMBED · AI.GENERATE_EMBEDDING

Managed scalar functions compose with SQL; optimized mode changes the execution strategy.



Lakehouse-native AI functions for scalable data transformation and enrichment, primarily targeting production batch pipelines

AI FUNCTION SURFACE

DOCUMENT PARSING

`ai_parse_document` · `ai_extract`

TEXT TASKS

`ai_classify` · `ai_summarize` · `ai_translate`

GENERAL INFERENCE

`ai_query` · `ai_gen`

SEARCH & FORECAST

`ai_similarity` · `ai_forecast` · `vector_search`

Lakehouse-native batch workflows span documents, scalar UDFs, vectors, and model serving.



LOTUS: An LLM-centric data processing framework that integrates LLMs as first-class operators into relational-style data workflows

AI FUNCTION SURFACE

Table 1: Summary of Semantic Operators. T denotes a table, X and Y denote arbitrary tuple types, $L[X]$ denotes a list of elements with type X , and A denotes the type of a particular column or attribute. l denotes a parameterized natural language expression ("langex" for short), which takes tuples as input and performs a function such as a predicate, an aggregation, a comparator, or a projection, depending on the operator's signature.

Operator	Description
$sem_filter(l: X \rightarrow Bool)$	Returns the tuples in a table that pass the provided langex predicate.
$sem_join(t: T, l: (X, Y) \rightarrow Bool)$	Joins a table against a second table t by keeping all pairs of tuples that pass the provided langex predicate.
$sem_sim_join(t: T, a_1: A_1, a_2: A_2, k: int)$	Performs a similarity join where each row of the source table is joined with the k most semantically similar tuples from the table t , using fields a_1 and a_2 as the left and right join keys, respectively.
$sem_agg(l: L[X] \rightarrow X)$	Performs an aggregation over the input tuples according to the langex, which specifies a commutative, associative aggregation function over a list of tuples.
$sem_topk(k: int, l: L[X] \rightarrow L[X])$	Ranks each tuple and returns the k best according to the langex, which specifies a ranking function that sorts a list of tuples.
$sem_map(l: X \rightarrow Y)$	Performs a projection, returning a new column, according to the provided langex.
$sem_extract(l: X \rightarrow Y)$	Performs a projection according to the langex, returning a column with a list of substrings from the input tuples.
$sem_cluster_by(C: int, a: A)$	Performs a semantic similarity clustering over column a to create C groups.
$sem_search(q: String, k: int, a: A)$	Performs a top- k search over column a using query q .
$sem_index(a: A, path: String)$	Creates similarity index for column a and saves to path.



Palimpzest

Palimpzest (a.k.a. Abacus): An optimizer-driven LLM data processing system that searches over plans to optimize the quality–cost–latency trade-off of AI-powered data pipelines.

AI FUNCTION SURFACE

Table 1: Semantic operators supported by ABACUS. In our implementation d is a (valid) JSON dictionary, but in principle d can be any serializable object. The $\cup$ symbol represents the union of output types. i is an integer index, P is a filter predicate, V is a vector database, and L is an integer limit. Aggregate includes group-by operations.

Operator Name	Symbol	Definition
Scan	ϕ	$\phi(i) \rightarrow d$
Map	μ	$\mu(d) \rightarrow d' \cup [d', d'', \dots]$
Filter	σ	$\sigma(d, P) \rightarrow d \cup \emptyset$
Retrieve	ρ	$\rho(d, V) \rightarrow d'$
Project	π	$\pi(d) \rightarrow d' \subseteq d$
Aggregate	α	$\alpha([d', d'', \dots]) \rightarrow \mathbb{R} \cup d$
Limit	λ	$\lambda([d', d'', \dots], L) \rightarrow [d', \dots, d^L]$



DocETL

DocETL: A document-centric ETL framework for processing large-scale document collections using LLM operators.

AI FUNCTION SURFACE

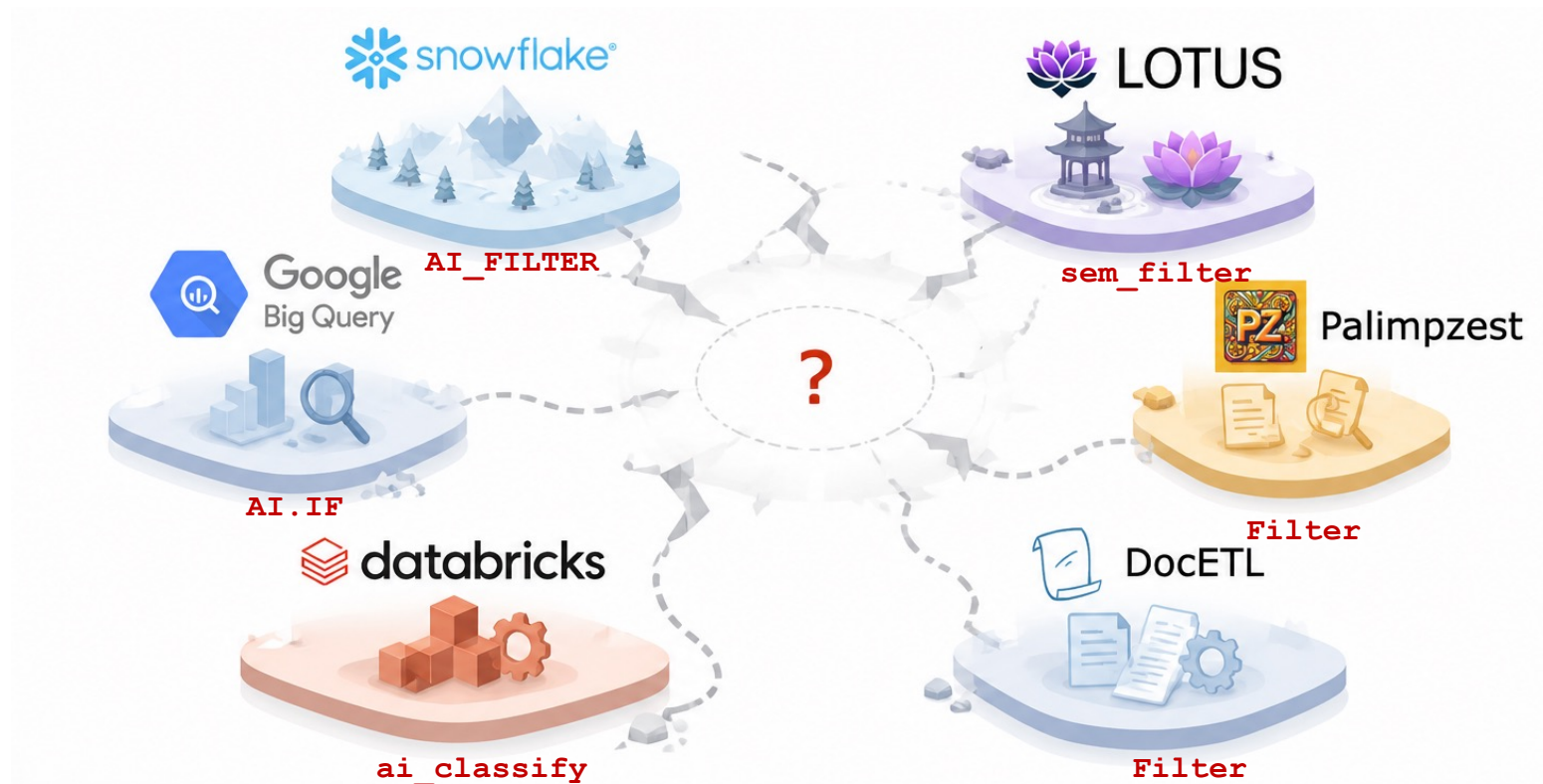
Table 1: DocETL’s operator suite, divided into operators that leverage LLMs for semantic processing and auxiliary operators (*) that handle data manipulation. For each operator, we show the required user configuration and a high-level description of its functionality.

Operator	User Configuration	Description
Map	Prompt, output schema	Uses an LLM to execute a transformation per document, adding resulting new keys to the schema (and optionally omitting existing ones).
Parallel Map	Multiple prompts, output schemas	Uses an LLM to execute multiple independent transformations on each document in parallel, adding the new keys to the schema.
Reduce	Group-by keys, prompt, output schema	Uses an LLM to aggregate groups of documents sharing the same key values into one new document per distinct value.
Filter	Prompt returning boolean	Uses an LLM to evaluate a condition per document, retaining only those where the condition is true.
Resolve	Comparison prompt, resolution prompt	Uses an LLM to identify values for a given key(s) that fuzzily match across documents and generate canonical versions per group of values, replacing them in-place in the documents.
Equijoin	Comparison prompt	Uses an LLM to determine if pairs of documents from two datasets should be joined based on fuzzy/semantic matching of the corresponding keys.
Unnest*	Array/dict field to unnest	Flattens nested data structures by either creating separate documents from array elements or merging nested dictionary fields into parent documents.
Split*	Split key, chunk size	Divides documents into smaller chunks based on token count or other criteria, creating as many new docs as there are chunks.
Gather*	Context window configuration	Augments each chunk with context from surrounding chunks based on specified configuration (e.g., previous and next chunk counts), keeping the set of documents the same.

Current States

The operator designs in these systems have never been systematically reviewed, logically aligned or comprehensively evaluated, leaving **a fragmented landscape**.

⇒ Is there a unified view to align these current LLM operators?



Taxonomy Overview

Separate the semantic action, the data unit, and the prompt execution pattern.

1 · OPERATING INTENT

What the LLM is asked to do

2 · OPERAND GRANULARITY

cell · row · column · table

3 · IMPLEMENTATION VARIANT

ALL · ONE · SEMI · PAIR · SORT ·
SCORE

FIVE OPERATING INTENTS

FILTER
select a subset

MATCH
link two sets

IMPUTE
add information

CLUSTER
partition a set

ORDER
rank a set

Taxonomy Overview

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Filter	Row	Filter records by semantic conditions	Select (σ)	LLM-ALL BlendSQL: LLMValidate LLM-ONE Aryn-Sycamore: Filter ThalamusDB: NLFilter Palimpzest: Filter DocETL: Filter LOTUS-TAG: sem_filter
	Column	Find columns by semantic conditions	Project (π)	LLM-ALL LLM-SL C3 LLM-ONE —
	Table	Identify tables by semantic conditions	—	LLM-ALL LLM-SL C3 LLM-ONE —
Match	Cell	Establish a semantic matching across two sets of cells to support table joining	Join ($\bowtie$)	LLM-ALL ThalamusDB: NLJoin (Batch) LLM-ONE ThalamusDB: NLJoin (Nested-Loop) Palimpzest: Join DocETL: Merge LLM-SEMI BlendSQL: LLMJoin LOTUS-TAG: sem_join
	Row	Find records with semantic connections, such as entity resolution (matching)	—	LLM-ALL ComEM ($m:n$) LLM-ONE DataWrangle ComEM (1:1) LLM-SEMI ComEM (1:n)
	Column	Identify semantically joinable columns on different tables	—	LLM-ALL — LLM-ONE — LLM-SEMI —
Impute	Cell	Infill missing cell values	—	LLM-ALL — LLM-ONE LLM-Preprocessor UniDM: Imputation
	Row	Insert new records into databases	—	LLM-ALL — LLM-ONE —
	Column	Extend tables with new columns with additional information	—	LLM-ALL Binder: f_col BlendSQL: LLMMap UniDM: Transform LLM-Relationalization LLM-ONE SUQL: Answer Aryn-Sycamore: Map HQDL: Generate Palimpzest: Map DocETL: Map LOTUS-TAG: sem_map LLM-GDO
Group	Row	Cluster records by semantic conditions	GroupBy (γ)	LLM-ALL LOTUS-TAG: sem_cluster.by LLM-ONE —
	Column	Organize columns by semantics, enabling table segmentation	—	LLM-ALL — LLM-ONE —
	Table	Organize tables by semantics, serving schema hierarchies	—	LLM-ALL — LLM-ONE —
Order	Row	Rank records by knowledge specified in semantic metrics	OrderBy (τ)	LLM-ALL — LLM-PAIR LOTUS-TAG: sem_topk (naive) LLM-SORT LOTUS-TAG: sem_topk (heap) LLM-SCORE —

LRO #1: Filter

WHAT

■ **Filter:** To semantically filter a subset of rows/columns/tables

■ **Common Operands:**

Row-level: Filter “technical companies” in Table (b)

Column/table-level: Identify tables and columns related to “company boss”

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Filter	Row	Filter records by semantic conditions	Select (σ)	LLM-ALL BlendSQL: LLMValidate LLM-ONE Aryn-Sycamore: Filter ThalamusDB: NLFilter Palimpzest: Filter DocETL: Filter LOTUS-TAG: sem.filter
	Column	Find columns by semantic conditions	Project (π)	LLM-ALL LLM-SL C3 LLM-ONE —
	Table	Identify tables by semantic conditions	—	LLM-ALL LLM-SL C3 LLM-ONE —

LRO #1: Filter

HOW

■ Implementation Variants:

LLM-ALL: To leverage batching, multiple candidates are provided to LLMs at once, and LLMs return the filtered ones

LLM-ONE: LLMs judge one candidate at one time, and return yes or no

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Filter	Row	Filter records by semantic conditions	Select (σ)	LLM-ALL BlendSQL: LLMValidate LLM-ONE Aryn-Sycamore: Filter ThalamusDB: NLFilter Palimpzest: Filter DocETL: Filter LOTUS-TAG: sem_filter
	Column	Find columns by semantic conditions	Project (π)	LLM-ALL LLM-SL C3 LLM-ONE —
	Table	Identify tables by semantic conditions	—	LLM-ALL LLM-SL C3 LLM-ONE —

LRO #2: Match

WHAT

- **Match:** To establish a matching relationship between two sets, similar to JOIN
- **Common Operands:**
 - Cell-level: Join Table (a) and Table (b) on “Company Name” and “Enterprise”
 - Row-level: Match identical companies
 - Column-level: Link semantically identical columns between two tables

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Match	Cell	Establish a semantic matching across two sets of cells to support table joining	Join ($\bowtie$)	LLM-ALL ThalamusDB: NLJoin (Batch) LLM-ONE ThalamusDB: NLJoin (Nested-Loop) Palimpzest:Join DocETL: Merge LLM-SEMI BlendSQL: LLMJoin LOTUS-TAG: sem_join
	Row	Find records with semantic connections, such as entity resolution (matching)	—	LLM-ALL ComEM ($m:n$) LLM-ONE DataWrangle ComEM (1:1) LLM-SEMI ComEM (1:n)
	Column	Identify semantically joinable columns on different tables	—	LLM-ALL — LLM-ONE — LLM-SEMI —

LRO #2: Match

HOW

■ Implementation Variants:

LLM-ALL: many-to-many matching by submitting all candidates from two sets to LLMs

LLM-ONE: iterative one-to-one matching by enumerating candidate pairs

LLM-SEMI: one-to-many matching, for each item in the left set, prompt with the whole right set

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Match	Cell	Establish a semantic matching across two sets of cells to support table joining	Join ($\bowtie$)	LLM-ALL ThalamusDB: NLJoin (Batch) LLM-ONE ThalamusDB: NLJoin (Nested-Loop) Palimpzest:Join DocETL: Merge LLM-SEMI BlendSQL: LLMJoin LOTUS-TAG: sem_join
	Row	Find records with semantic connections, such as entity resolution (matching)	—	LLM-ALL ComEM ($m:n$) LLM-ONE DataWrangle ComEM (1:1) LLM-SEMI ComEM (1:n)
	Column	Identify semantically joinable columns on different tables	—	LLM-ALL — LLM-ONE — LLM-SEMI —

LRO #3: Impute

WHAT

- **Impute:** To inject new information to the relation via LLMs
- **Common Operands:**
 - Cell-level: Infer the gender of Clara Smith
 - Column-level: Add a new column indicating the country of each enterprise
 - Row-level: Insert a new company OpenAI

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Impute	Cell	Infill missing cell values	—	LLM-ALL — LLM-ONE LLM-Preprocessor UniDM:Imputation
	Row	Insert new records into databases	—	LLM-ALL — LLM-ONE —
	Column	Extend tables with new columns with additional information	—	LLM-ALL Binder: f_col BlendSQL: LLMap UniDM: Transform LLM-Relationalization LLM-ONE SUQL: Answer Aryn-Sycamore: Map HQDL: Generate Palimpzest: Map DocETL: Map LOTUS-TAG: sem_map LLM-GDO

LRO #3: Impute

HOW

■ Implementation Variants:

LLM-ALL: LLMs impute all cells at one time, and return the complete imputed table

LLM-ONE: LLMs impute each candidate element independently through multiple independent invocations

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Impute	Cell	Infill missing cell values	—	LLM-ALL — LLM-ONE LLM-Preprocessor UniDM:Imputation
	Row	Insert new records into databases	—	LLM-ALL — LLM-ONE —
	Column	Extend tables with new columns with additional information	—	LLM-ALL Binder: f_col BlendSQL: LLMap UniDM: Transform LLM-Relationalization LLM-ONE SUQL: Answer Aryn-Sycamore: Map HQDL: Generate Palimpzest: Map DocETL: Map LOTUS-TAG: sem_map LLM-GDO

LRO #4: Cluster

WHAT

- Cluster: To semantically partition the candidate set

- **Common Operands:**

Row-level: Cluster enterprises in Table (b) into technology and retail

Column-level: Cluster similar types of columns

Table-level: Organize tables into schema hierarchies

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations			
Group	Row	Cluster records by semantic conditions	GroupBy (γ)	LLM-ALL	LOTUS-TAG: sem_cluster_by	LLM-ONE	—
	Column	Organize columns by semantics, enabling table segmentation	—	LLM-ALL	—	LLM-ONE	—
	Table	Organize tables by semantics, serving schema hierarchies	—	LLM-ALL	—	LLM-ONE	—

LRO #4: Cluster

HOW

■ Implementation Variants:

LLM-ALL: via a single prompt with full visibility of all elements

LLM-ONE: LLMs iteratively assign each element to an existing or a new cluster

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations		
Group	Row	Cluster records by semantic conditions	GroupBy (γ)	LLM-ALL	LOTUS-TAG: <code>sem_cluster_by</code>	LLM-ONE —
	Column	Organize columns by semantics, enabling table segmentation	—	LLM-ALL —	LLM-ONE	—
	Table	Organize tables by semantics, serving schema hierarchies	—	LLM-ALL —	LLM-ONE	—

LRO #5: Order

WHAT + HOW

■ **Order:** To semantically rank tuples

■ **Common Operands:**

Row-level: Rank the conferees in Table (a) by their social influence

Intent	Operand	Functionality	Primitive	Existing LROs and Implementations
Order	Row	Rank records by knowledge specified in semantic metrics	OrderBy (τ)	LLM-ALL — LLM-PAIR LOTUS-TAG: sem_topk (naive) LLM-SORT LOTUS-TAG: sem_topk (heap) LLM-SCORE —

■ **Implementation Variants:**

LLM-ALL: batching all tuples via a single prompt

LLM-PAIR: pair-wise LLM comparison

LLM-SORT: a quick sort, but replacing the conventional numerical comparison to LLM semantical comparison

LLM-SCORE: LLMs assign a numerical score in $[0, 100]$ to each row, and rank

Benchmark & Single-LRO Evaluation

Benchmark Aim

A useful benchmark must satisfy all four requirements at once.

01

NATIVE TO RELATIONS

Relational input and relational output

02

EXPLICIT LRO INVOLVEMENT

Every query exercises at least one LRO

03

ISOLATED COVERAGE

Cover every intent × operand combination

04

COMPOSED DIFFICULTY

Stratified multi-LRO query plans

Existing benchmarks miss at least one requirement.

Benchmark	Native to Relations	Diversity of Queries	Single-LRO Coverage	Multi-LRO Stratification
Fact Check	✗	✗	✗	✗
QA Bench	✗	✗	✗	✗
Text-to-SQL	✓	✗	✗	✗
SWAN [59]	✓	✗	✗	✗
TAG [5]	✓	✗	✗	✗
SemBench [26]	✗	✗	✗	✗
LROBENCH (Ours)	✓	✓	✓	✓

Current Benchmarks are Insufficient

GAP · 1/2

Unfortunately, **existing benchmarks** fail to meet our requirements

- **Fact Check Benchmark:** assess textual claims against evidence in tables, and return yes/no

Limit: Inputs, outputs are not native to relations

- **QA Bench Benchmark:** answer declarative questions over relational tables

Limit: Outputs are not native to relations

- **Text-to-SQL Benchmark:** translate NL queries into SQL statements

Limit: Do not involve LROs

Benchmark	Native to Relations	Diversity of Queries	Single-LRO Coverage	Multi-LRO Stratification
Fact Check	✗	✗	✗	✗
QA Bench	✗	✗	✗	✗
Text-to-SQL	✓	✗	✗	✗
SWAN [59]	✓	✗	✗	✗
TAG [5]	✓	✗	✗	✗
SemBench [26]	✗	✗	✗	✗
LroBENCH (Ours)	✓	✓	✓	✓

Semantic-Query Benchmarks Still Leave Coverage Gaps

GAP · 2/2

Some existing **beyond-database query** benchmarks align with our scope to some extent

- **TAG, SWAN**: extend SQL with LROs, and design beyond-database queries to test their capabilities

Limit: Only a narrow subset of LROs are considered

- **SemBench**: consider all LROs, and evaluate semantic query process engines

Limit: Inputs not native to relations; neglecting relational operands; few multi-LRO queries and no difficulty stratification

Benchmark	Native to Relations	Diversity of Queries	Single-LRO Coverage	Multi-LRO Stratification
Fact Check	✗	✗	✗	✗
QA Bench	✗	✗	✗	✗
Text-to-SQL	✓	✗	✗	✗
SWAN [59]	✓	✗	✗	✗
TAG [5]	✓	✗	✗	✗
SemBench [26]	✗	✗	✗	✗
LroBENCH (Ours)	✓	✓	✓	✓

LROBench: Overview

- LROBench consists of:
 - ❑ 27 real-world databases from open-source benchmarks, spanning over 10 domains
 - ❑ 290 single-LRO queries that isolate one target LRO
 - ❑ 60 multi-LRO queries that require multiple LROs to solve more challenging tasks

An example single-LRO query

[Question *Q*] Identify and select the columns relevant to the restaurant atmosphere.
[Plan *P*] `LLM_FILTER`(Restaurants, 'column', 'It is related to the atmosphere.');

[Ground Truth *R*] A one-column result table containing only the "Description" column.

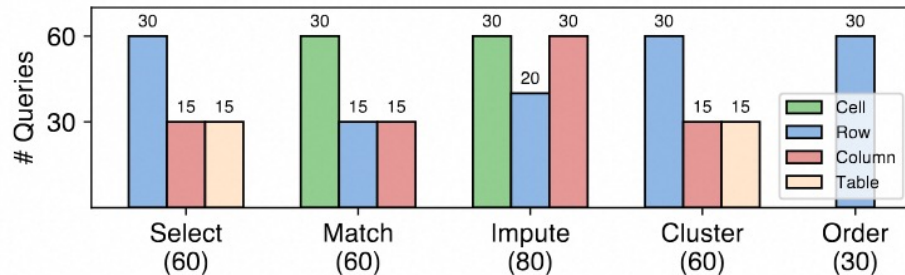
An example multi-LRO query

[Question *Q*] Find the restaurant best suits Asian tastes in the Bay Area.
[Plan *P*] `SELECT Name FROM Restaurants WHERE LLM_FILTER('row', 'Location is in Bay Area.')`
`ORDER BY LLM_ORDER('row', 'Rank by appeal to Asian tastes from best to worst.')` `LIMIT 1;`
[Ground Truth *R*] A one-cell table "Alley Wok".

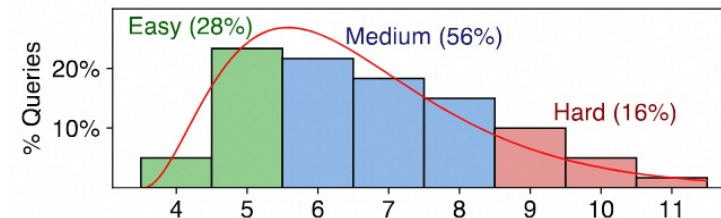
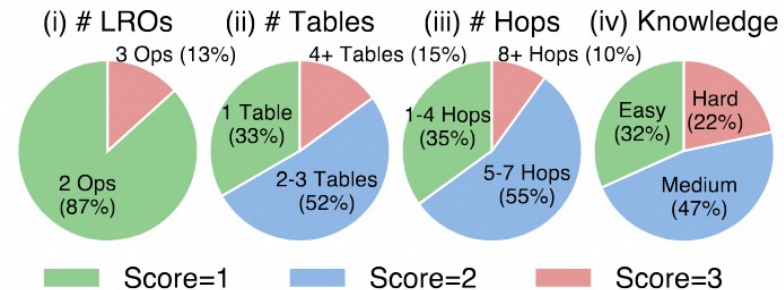
LROBench: Overview

■ LROBench distribution

- ❑ 290 single-LRO queries with full coverage of the operator-operand combinations in the taxonomy
- ❑ 60 multi-LRO queries with near-normal scoring distribution



single-LRO query counts by operators and operands



multi-LRO query scoring dimensions and distributions

Single-LRO Evaluation: Setup

First, we ask *How do different LRO implementations impact LRO performance?*

■ Setup

- Query set: 290 single-LRO queries in LROBench
- Baselines: Different implementation strategies such as LLM-ALL, LLM-ONE...
- Base models: GPT-5, Qwen3-Max and Claude-Sonnet-4.5

Query example

[Question Q] Identify and select the columns relevant to the restaurant atmosphere.

[Plan P] `LLM_FILTER`(Restaurants, 'column', 'It is related to the atmosphere.');

[Ground Truth R] A one-column result table containing only the "Description" column.

Filter & Match: Context Decides the Best Variant

1. Filter

- **Metric:** Precision, recall, F1
- **Key Take-aways:** Row-wise LLM-ONE is a strong default; for column/table operands, prefer LLM-ALL. Strong long-context models can also make row-wise LLM-ALL competitive and cheaper.

2. Match

- **Metric:** Precision, recall, F1
- **Key Take-aways:** LLM-ALL leads on cell/column; for row-wise, LLM-SEMI matches or beats it and is the recommended default. Use smart batching when one global prompt becomes too large.

Table 3: Key Filter results across base models.

Operand	Impl	GPT-5		Qwen3-Max		Claude-Sonnet-4.5	
		F1	Cost (\$)	F1	Cost (\$)	F1	Cost (\$)
Row	LLM-ALL	0.945	0.005	0.917	0.004	0.846	0.009
	LLM-ONE	<u>0.940</u>	0.025	0.932	0.026	0.937	0.078
Column	LLM-ALL	0.891	0.002	<u>0.874</u>	0.002	0.865	0.005
	LLM-ONE	0.849	0.014	0.869	0.015	0.771	0.043
Table	LLM-ALL	0.921	0.009	<u>0.939</u>	0.010	0.943	0.024
	LLM-ONE	0.654	0.020	0.612	0.021	0.617	0.060

Table 4: Key Match results across base models.

Operand	Impl	GPT-5		Qwen3-Max		Claude-Sonnet-4.5	
		F1	Cost (\$)	F1	Cost (\$)	F1	Cost (\$)
Cell	LLM-ALL	0.843	0.004	0.758	0.003	<u>0.821</u>	0.008
	LLM-ONE	0.802	0.174	0.704	0.185	0.753	0.513
	LLM-SEMI	0.789	0.014	0.729	0.014	0.756	0.047
Row	LLM-ALL	0.961	0.029	0.735	0.024	0.929	0.058
	LLM-ONE	0.829	0.338	0.862	0.335	0.797	0.946
	LLM-SEMI	0.934	0.085	0.922	0.079	<u>0.941</u>	0.200
Column	LLM-ALL	0.746	0.003	<u>0.735</u>	0.003	0.726	0.008
	LLM-ONE	0.591	0.154	0.415	0.166	0.571	0.484
	LLM-SEMI	0.705	0.017	0.623	0.019	0.637	0.050

Impute & Cluster: Global Context Helps Selectively

3. Impute

- **Metric:** Exact match, LLM judge score
- **Key Take-aways:** When generating columns, LLM-ONE is more accurate. For cell and row-wise imputation, LLM-ALL with sufficient context is preferred

4. Cluster

- **Metric:** ARI, NMI
- **Key Take-aways:** LLM-ALL with global context is recommended across the evaluated operands

Table 5: Key Impute results across base models.

Operand	Impl	GPT-5 Judge Score	Qwen3-Max Judge Score	Claude-Sonnet-4.5 Judge Score
Cell	LLM-ALL	0.886	0.820	<u>0.864</u>
	LLM-ONE	0.848	0.824	0.833
Column	LLM-ALL	0.816	0.737	0.834
	LLM-ONE	<u>0.838</u>	0.793	0.859
Row	LLM-ONE w/o EX	0.607	0.664	0.561
	LLM-ONE w/ EX	<u>0.724</u>	0.751	0.714

Table 6: Key Cluster results across base models.

Operand	Impl	GPT-5		Qwen3-Max		Claude-Sonnet-4.5	
		ARI	NMI	ARI	NMI	ARI	NMI
Row	LLM-ALL	0.723	0.801	<u>0.720</u>	<u>0.778</u>	0.675	0.763
	LLM-ONE	0.694	0.759	0.658	0.727	0.696	0.756
Column	LLM-ALL	<u>0.812</u>	<u>0.877</u>	0.751	0.835	0.848	0.897
	LLM-ONE	0.604	0.721	0.493	0.641	0.610	0.726
Table	LLM-ALL	0.721	0.837	0.627	0.783	<u>0.716</u>	<u>0.830</u>
	LLM-ONE	0.668	0.801	0.630	0.780	0.647	0.788

Order: LLM-ALL Is the Robust Default

5. Order

- **Metric:** Top-k hit rate, Kendall's τ
- **Key Take-aways:**
 - LLM-ALL is strongly recommended.
 - Both LLM-PAIR and LLM-SORT methods are vulnerable to inconsistent judgments.
 - LLM-SCORE, which lacks a reasonable sense of scores, is not recommended.

Table 7: Row-wise Order results across base models.

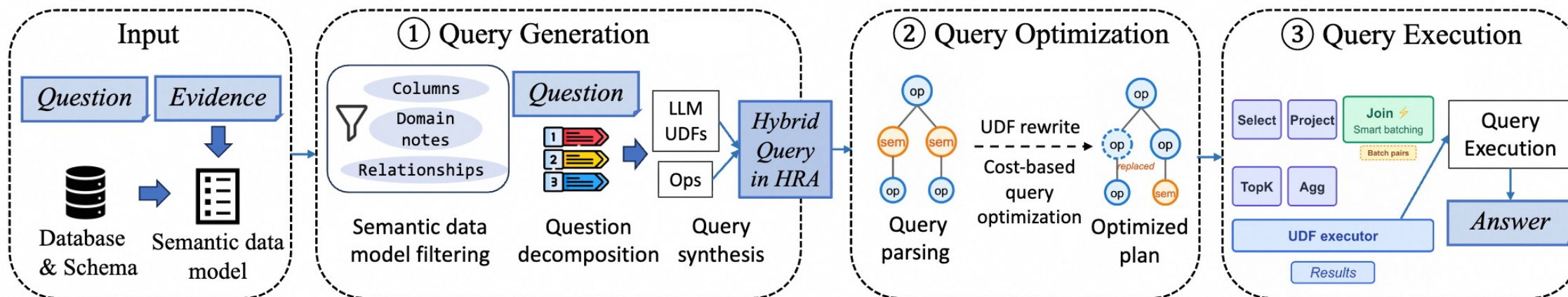
Impl	GPT-5		Qwen3-Max		Claude-Sonnet-4.5	
	HR@k	τ	HR@k	τ	HR@k	τ
LLM-ALL	0.941	0.937	0.952	0.884	<u>0.949</u>	0.865
LLM-PAIR	0.929	0.907	0.921	0.892	0.910	<u>0.914</u>
LLM-SORT	0.896	0.881	0.941	0.884	0.899	<u>0.871</u>
LLM-SCORE	0.776	0.658	0.805	0.588	0.798	0.706

More calls do not guarantee a more reliable ranking.

From Operator To System

Query Optimization for LLM-Enhanced SQL

Sema-SQL builds all three: **generate the plan** · **optimize it** · **execute it**



1 · Generate the query plan

- Retrieve relevant semantic context
- Decompose the user question
- Synthesize relational and LLM operators into an HRA plan

2 · Optimize the plan

- Delay LLM evaluation until it is necessary
- Rewrite eligible LLM UDFs as SQL
- Check equivalence and reorder operators by cost

3 · Execute the optimized plan

- Deduplicate inputs
- Batch semantic joins intelligently
- Execute relational and LLM operators efficiently

Natural Language as a Query Language

SQL is an interface for specialists. The user's question should be the query language—for everyone.

The Promise

"How many test takers attend schools in counties with populations above 2M?"

→ **Answer** — *no SQL, no code: anyone who can ask can query*

From Query to Answer

LLM planning → sandboxed execution → answer

Planning

What the LLM Receives

- What the user's query intent is
- What the user's data means
- What each available operator can do

What the LLM Produces

An executable plan for the whole query—one the engine can ground, optimize, verify, and execute.

Teaching the LLM to plan—and building the system that can safely execute that plan.

Operators for LLM-Enhanced Relational Algebra

Operators	Semantic capability	Example signature → return type
Projection	Derive a typed attribute from unstructured text	<code>llm_extract</code> (body, 'root cause') → TEXT
Selection	Evaluate a predicate no stored column encodes	<code>llm_filter</code> (body, 'revenue impact') → BOOL
Join	Match records when no shared key exists	<code>llm_match</code> (a.name, t.customer) → BOOL
Top-K	Rank candidates using semantic judgment	<code>llm_cmp</code> (a.body, b.body, 'more severe') → BOOL
Aggregation	Produce one grounded answer for each group	<code>llm_summarize</code> (body, 'root cause') → TEXT

Explicit return types make semantic operators plannable: the optimizer can move, batch, or rewrite them.

Embedding LLM Calls into Different Languages

```
scores_df = pd.read_csv("satscores.csv")
schools_df = pd.read_csv("schools.csv")
unique_counties = pd.DataFrame(schools_df["County"].unique(), columns=["
County"])
unique_counties = unique_counties.sem_map(
    "What is the population of {County} in California? Answer with only the
    number without commas. Respond with your best guess."
)
counties_over_2m = set()
for _, row in unique_counties.iterrows():
    try:
        if int(re.findall(r"\d+", row._map)[-1]) > 2000000:
            counties_over_2m.add(row.County)
    except:
        pass
schools_df = schools_df[schools_df["County"].isin(counties_over_2m)]
merged = pd.merge(scores_df, schools_df, left_on="cds", right_on="CDSCode")
prediction = int(merged["NumTstTskr"].sum())
```

(a) LOTUS program. [Patel et al., VLDB 2024]

```
YSUM(NumTstTskr)→total_test_takers
(satscores ⋈cds = CDSCode
  (σpopulation > 2000000
    (ΠExtractPopulation(County)→population
      (schools))))
```

(b) HRA representation.

Semantic operators inside analytical programs

LLM calls can appear as ordinary pipeline steps—for example, `sem_map` in a pandas-style program.

Every call has a relational meaning

extract is projection π , filter is selection σ , match is join $\bowtie$ — the program in (a) is the algebra in (b).

Hybrid Relational Algebra

Relational algebra + LLM operators: the query specifies **what** to compute; the engine determines **how** to execute it.

From Database DDL to a Semantic Data Model

Database DDL

```
CREATE TABLE schools (  
  CDSCode      TEXT,  
  County       TEXT,  
  NumTstTskr  INT  
);
```

Structure without meaning: What does “test taker” mean? How do these tables relate?

Semantic Data Model

```
tables:  
  schools:  
    description: California public schools  
    columns:  
      CDSCode:      school identifier  
      County:       county name  
      NumTstTskr:   number of SAT test takers  
relationships:  
  - schools.CDSCode = satscores.cds  
  
domain_notes:  
  - “test takers” means NumTstTskr  
  - population is not stored anywhere
```

Semantic Data Model Filtering: pass only the tables, columns, relationships, and domain notes relevant to the question into the generation prompt.

The semantic data model tells the LLM what each column means, how tables connect, and which external facts the question requires.

Example Retrieval: Aligning the Question and the SQL

*Examples teach the generator in context. DAIL-SQL explores the best example **selection**, **schema representation**, and **prompt organization**.*

Similar Question?

Abstract away entities and values; compare **question skeletons**.

“how many [X] at [Y] in [Z] over [N]?”

Similar SQL?

Draft SQL for the new question; compare **query skeletons**.

SELECT COUNT(*) ... JOIN ... WHERE [cond]

Each design choice is benchmarked across both open-source and proprietary LLMs.

*Examples show the model what a good query looks like, next **guide it to construct one step by step**.*

Query Generation: From One-Shot to Guided

Decomposing planning into explicit steps produces more accurate, verifiable query plans.

One-Shot Generation

```
[semantic data model]
[operator catalog] [question]
```

Write the HRA query.

Common Failures

- Hallucinated columns or incorrect join paths
- Semantic operators skipped—or used where SQL is sufficient
- Untyped calls that the parser cannot validate

Guided, Step-by-Step Generation

```
[semantic data model]
[operator catalog] [question]
```

[Decomposition] Build the query, one step at a time:

- [operator selection] pick one operator, bind real columns
- [justification] justify the selection of each operator
- [parameter checking] emit its typed signature

Stop when the query intent is met.

Example of Decomposition It Produces

- **Semantic projection** — derive population per county; “population” not in DB
- **Selection** — population > 2,000,000
- **Join** — schools ⋈ satscores on CDSCode
- **Aggregation** — SUM(NumTstTkr)

Compile the steps into an HRA plan

One large leap becomes **four small, auditable decisions**.

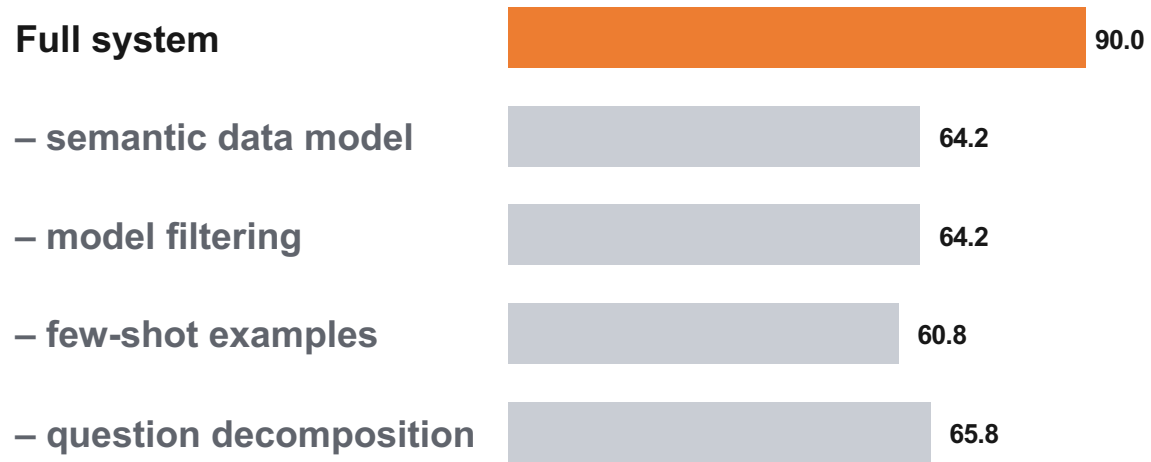
Experiments: Structured Generation

Evaluation Setup

TAG [Biswal et al., CIDR2025] — BIRD databases, with questions rewritten to require semantic operators.

Metric: query-generation accuracy—the share of questions that produce a correct plan.

LLM: Claude Sonnet 4.5.



Ablation — Query-Generation Accuracy (%)

What makes it reliable?

- Parser feedback catches syntax and schema errors.
- The semantic data model reduces column hallucination.
- Few-shot examples stabilize types and multi-operator plans.
- Question decomposition prevents missing operator choices.

Removing any single component drops accuracy from 90% to the low 60s—each one protects against a distinct failure mode.

AmbiSQL: A Taxonomy of Text-to-SQL Ambiguity

*“How many Formula one drivers born after **the end of the Vietnam War** have been **ranked 2**?”*

Text-to-SQL maps language onto two worlds: the **database schema**, and the **model’s knowledge**. Ambiguity can arise in either mapping; resolution begins by identifying which one.

DATABASE-SOURCED · What database element did the user mean?

AmbiSchema Which table or column?

“oldest user” — age, or signup date?

AmbiValue Does the literal exist as written?

asking “New York City” when the table stores “NYC”

AmbiIntent Which operation is intended?

“by registration date” — order, group, or filter?

LLM-SOURCED · What must the model reason about?

AmbiSource Is the answer in the database or external knowledge?

“taller than Michael Jordan” — whose height, and from which source?

AmbiContext Is there enough context to reason correctly?

“exchange rate” — between which currencies, on what date?

AmbiFallacy Does the premise hold?

asking about an event that never took place

AmbiRef At what level of granularity should the reference be interpreted?

“after the 2018 World Cup” — after the final, or after that year?

Naming the ambiguity makes the question answerable: each type becomes a concrete disambiguation prompt that a non-expert can resolve.

AmbiSQL: Plug-In Intent Clarification

What the User Sees · No SQL, No Schema

“the end of the Vietnam War” means:

- **March 29, 1973 — the ceasefire**
 - April 30, 1975 — the fall of Saigon
 - Abstain / Other

“ranked 2” refers to:

- **driverStandings.position**
 - a rank column on another table
 - Abstain / Other

What Changes in SQL · The Invisible Difference

Date comparison

Guessed `born > '1975-04-30'`

Clarified `born > '1973-03-29'`

Column filter

Guessed `rank = 2`

Clarified `position = 2`

Both queries run and return a number—only one answers the intended question.

Where It Fits · Before Any Generator

Questions → **AmbiSQL** detect · ask · rewrite → **clarified question** → any generator

As middleware, AmbiSQL resolves intent before planning—so any Text-to-SQL or agentic system inherits the improvement. [Demo of AmbiSQL](#)

System Optimization

Optimizing the Cost of LLM-Enhanced Queries

Objective

$$\text{cost}(Q) = \text{cost_SQL}(Q) + \text{cost_LLM}(Q)$$

- Minimize this cost over all equivalent query plans.
- LLM calls dominate the budget: a single model invocation can cost orders of magnitude more than a relational operation.

Logical Optimization: Rewrite the Plan, Preserve the Answer

1. Push LLM Operators up

Apply filters and joins first, then invoke the LLM on the reduced input.

2. Compile to SQL

When an LLM operator is deterministic, rewrite it as a standard SQL predicate—eliminating the model call.

```
'in the Bay Area' → County IN ('Alameda','Marin',...)
```

Physical Optimization: Execute the Same Plan More Cheaply

1. Semantic Batching

Make one call per batch pair rather than per tuple pair—yielding 5–300× fewer LLM invocations.

2. Model Cascades

Use a low-cost model first; escalate only uncertain rows to a stronger model.

*The logical layer decides **where** each LLM operator sits in the plan. The physical layer decides **how** each call runs.*

Measured results: 28% lower execution time and 21% fewer tokens on optimized queries.

Minimize the Rows the Model Must See

Question: “How many Asian drivers competed in the 2008 Malaysian Grand Prix?”

Plan 1 · Apply LLM Selection First

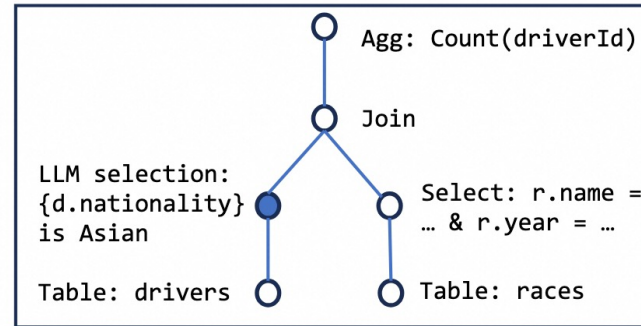
$\sigma_{llm(driver\ is\ Asian)}$ $\bowtie$ results $\bowtie$ races

The LLM evaluates every driver—hundreds of calls—although most rows are later eliminated by the join.

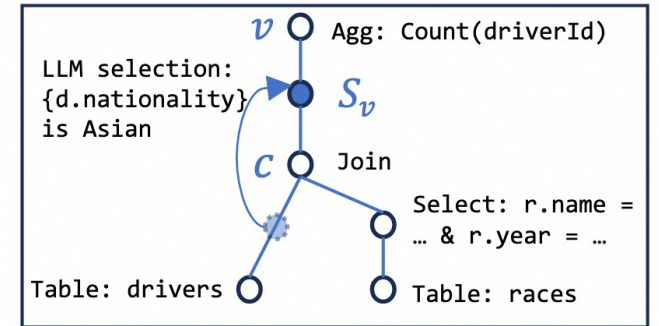
Plan 2 · Apply Cheap Filters First

$\sigma_{race = 2008\ Malaysia}$ $\bowtie$ results $\rightarrow \sigma_{llm(...)}$

The LLM evaluates only one race’s grid—about 20 calls—with the same answer.



(1): Original query plan



(2): Query plan with lazy LLM evaluation

Push expensive semantic operators below selective predicates: **fewer model calls, equivalent result.**

SEMA-SQL searches valid operator placements and verifies equivalence through symbolic execution.

Smart Batching Optimizes Semantic Joins

Semantic joins are expensive when evaluated pair by pair. Smart batching replaces many LLM calls with a small number of batched calls.

Join inputs — the same companies, written differently

CustomerID	CompanyName
C101	International Business Machine
C102	Microsoft Corporation
C103	Google LLC
C104	Proctor & Gamble Co.
C105	Deloitte Consulting LLP

SaleID	ClientName	Revenue
S001	IBM	500,000
S002	Microsoft Corp.	1,200,000
S003	Google Inc.	700,000
S004	P&G	1,000,000
S005	Reckitt	450,000

Two ways to execute the same join

Nested Loop

$|K_1| \times |K_2|$ LLM calls

e.g. $5 \times 5 = 25$ calls

Smart Batching

Group keys so that one call returns all matches for a batch pair.

e.g. one sizing call + one matching call = 2 calls

The Two Types of Calls

Call 1 · Sizing probe the task and choose a batch size the model can handle reliably.

Call 2 · Matching read two key lists and return every matching pair at once.

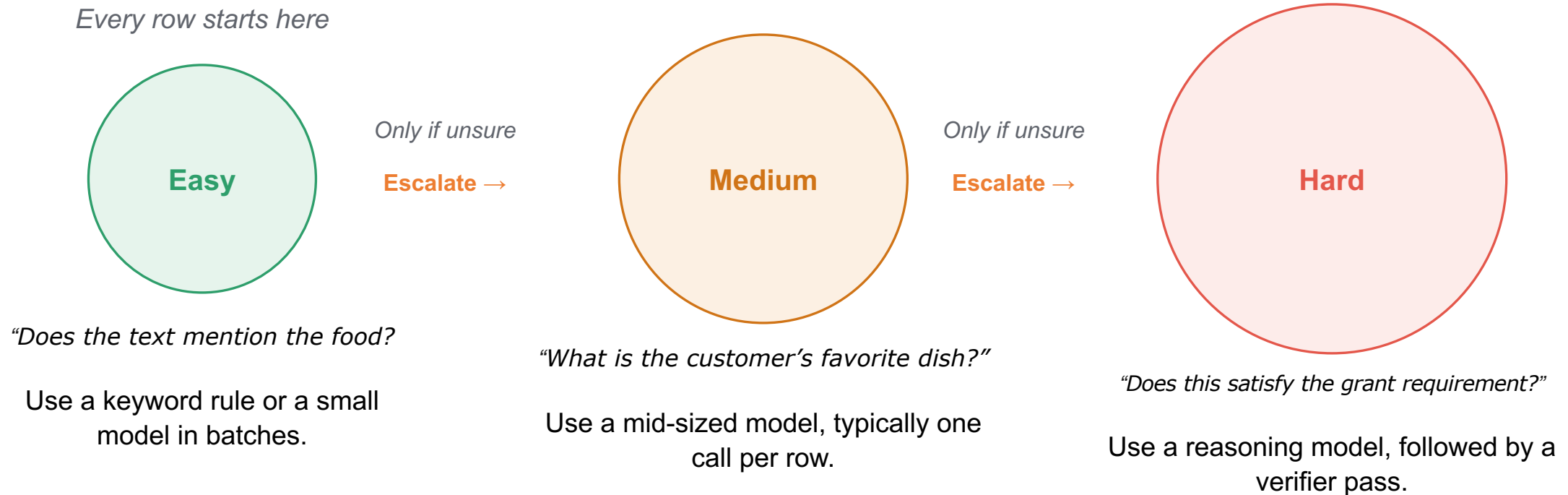
Example:

IBM ↔ International Business Machine · P&G ↔ Proctor & Gamble Co. · ...

Across semantic joins, smart batching achieves: **93.3% fewer LLM calls at 100% execution accuracy.**

Route Each Row to the Right Model

A single semantic predicate can vary in difficulty—**easy rows should not incur reasoning-model costs.**



Most rows are easy — Difficulty-aware routing keeps the average cost near the cheapest tier

The logical operator stays the same; choosing the model is a physical execution decision.

Multi-LRO System Evaluation

Multi-LRO System Evaluation Setup

- Evaluation Aim: *How should multi-LRO systems be designed to incorporate operator-level best practices, and do existing systems do so effectively?*
- Setup
 - Query set: 60 multi-LRO queries in LROBench
 - Base models: GPT-5, Qwen3-Max and Claude-Sonnet-4.5
 - Metric: Exact Match (EM) between the output and ground-truth tables: a query is counted as correct only if the two tables are **identical**.

Query example

```
[Question Q] Find the restaurant best suits Asian tastes in the Bay Area.  
[Plan P] SELECT Name FROM Restaurants  
WHERE LLM_FILTER('row', 'Location is in Bay Area.')
```

```
ORDER BY LLM_ORDER('row', 'Rank by appeal to Asian tastes from best to worst.') LIMIT 1;  
[Ground Truth R] A one-cell table "Alley Wok".
```

Incomplete Stage of Current Systems

- Not all baseline systems explicitly support all five LROs in our benchmark. We report their LRO support as follows
 - In the worst case, the system is **completely incapable** of realizing the logic of the LRO

Half of the current systems fail to cover all queries

Table 10: LRO correspondences and benchmark query coverage of existing multi-LRO systems.

System	Select	Match	Impute	Cluster	Order	Query Coverage
Binder	f_col + <u>SELECT</u>	✗	f_col	f_col + <u>GROUPBY</u>	f_col + <u>ORDERBY</u>	33.3%
SUQL	Answer + <u>SELECT</u>	✗	Answer	Answer + <u>GROUPBY</u>	Answer + <u>ORDERBY</u>	75.0%
Aryn-Sycamore	Filter	✗	Map	Map + <u>GROUPBY</u>	Map + <u>ORDER</u>	33.3%
HQDL	Generate + <u>SELECT</u>	✗	Generate	Generate + <u>GROUPBY</u>	Generate + <u>ORDERBY</u>	75.0%
ThalamusDB	NLFILTER	NLJOIN	✗	✗	✗	41.7%
BlendSQL	LLMValidate	LLMJoin	LLMMap	LLMMap + <u>GROUPBY</u>	LLMMap + <u>ORDERBY</u>	100.0%
Palimpzest	Filter	Join	Map	Map + <u>GROUPBY</u>	Map + <u>ORDERBY</u>	100.0%
DocETL	Filter	Merge	Map	Map + <u>GROUPBY</u>	Map + <u>ORDERBY</u>	100.0%
LOTUS-TAG	sem_filter	sem_join	sem_map	sem_cluster_by	sem_topk	100.0%
Our Best Practices	Select	Match	Impute	Cluster	Order	100.0%

Early-Stage and Room for Improvement

- Automatic LRO planning remains **early-stage** and multi-LRO systems still have substantial room for improvement
 - ▣ LLM-planning systems perform poorly on our benchmark ($< 15\%$ accuracy) and fail on complex queries
 - ▣ Even with expert-annotated plans, most manual-planning systems (except Palimpzest) remain **below 55%**

Table 11: Performance of multi-LRO systems on multi-LRO benchmark.

Metric		LLM-Planning Systems			Manual-Planning Systems						Our Best Practices
		Binder	SUQL	Aryn-Sycamore	HQDL	ThalamusDB	BlendSQL	Palimpzest	DocETL	LOTUS-TAG	
GPT-5	Accuracy	0%	13.33%	0%	30.00%	33.33%	55.00%	43.33%	13.33%	50.00%	86.67%
	Cost (\$)	0.44	0.02	0.08	1.13	0.12	0.12	0.46	0.78	0.40	0.17
Qwen3-Max	Accuracy	1.67%	6.67%	0%	31.67%	15.00%	31.67%	70.00%	33.33%	38.33%	71.67%
	Cost (\$)	0.24	0.01	0.36	1.24	0.03	0.03	0.32	0.41	0.15	0.06
Claude-Sonnet-4.5	Accuracy	0%	6.67%	0%	33.33%	8.33%	35.00%	66.67%	40.00%	38.33%	76.67%
	Cost (\$)	0.91	0.03	0.05	2.42	0.16	0.10	0.80	1.45	0.35	0.24
Stratified Accuracy under GPT-5	Easy (n=17)	0%	23.53%	0%	35.29%	35.29%	70.59%	52.94%	5.88%	64.71%	100.00%
	Medium (n=33)	0%	12.12%	0%	30.30%	33.33%	54.55%	42.42%	18.18%	51.52%	81.82%
	Hard (n=10)	0%	0%	0%	20.00%	10.00%	30.00%	30.00%	10.00%	30.00%	80.00%

Specialized Beats One-Size-Fits-All

- LRO implementations that **follow our best practices** often perform better (Palimpzest)
- In contrast, **overly general, one-fits-all LROs** are neither effective nor cost-efficient (HQDL)

Table 11: Performance of multi-LRO systems on multi-LRO benchmark.

Metric		LLM-Planning Systems			Manual-Planning Systems						Our Best Practices
		Binder	SUQL	Aryn-Sycamore	HQDL	ThalamusDB	BlendSQL	Palimpzest	DocETL	LOTUS-TAG	
GPT-5	Accuracy	0%	13.33%	0%	30.00%	33.33%	55.00%	43.33%	13.33%	50.00%	86.67%
	Cost (\$)	0.44	0.02	0.08	1.13	0.12	0.12	0.46	0.78	0.40	0.17
Qwen3-Max	Accuracy	1.67%	6.67%	0%	31.67%	15.00%	31.67%	70.00%	33.33%	38.33%	71.67%
	Cost (\$)	0.24	0.01	0.36	1.24	0.03	0.03	0.32	0.41	0.15	0.06
Claude-Sonnet-4.5	Accuracy	0%	6.67%	0%	33.33%	8.33%	35.00%	66.67%	40.00%	38.33%	76.67%
	Cost (\$)	0.91	0.03	0.05	2.42	0.16	0.10	0.80	1.45	0.35	0.24
Stratified Accuracy under GPT-5	Easy (n=17)	0%	23.53%	0%	35.29%	35.29%	70.59%	52.94%	5.88%	64.71%	100.00%
	Medium (n=33)	0%	12.12%	0%	30.30%	33.33%	54.55%	42.42%	18.18%	51.52%	81.82%
	Hard (n=10)	0%	0%	0%	20.00%	10.00%	30.00%	30.00%	10.00%	30.00%	80.00%

Best Practices as Guidelines

- Our best-practice baseline consistently performs best and solves **up to 80%** of hard queries, suggesting it may serve as a useful guideline for future systems

Table 11: Performance of multi-LRO systems on multi-LRO benchmark.

Metric		LLM-Planning Systems			Manual-Planning Systems						Our Best Practices
		Binder	SUQL	Aryn-Sycamore	HQDL	ThalamusDB	BlendSQL	Palimpzest	DocETL	LOTUS-TAG	
GPT-5	Accuracy	0%	13.33%	0%	30.00%	33.33%	55.00%	43.33%	13.33%	50.00%	86.67%
	Cost (\$)	0.44	0.02	0.08	1.13	0.12	0.12	0.46	0.78	0.40	0.17
Qwen3-Max	Accuracy	1.67%	6.67%	0%	31.67%	15.00%	31.67%	70.00%	33.33%	38.33%	71.67%
	Cost (\$)	0.24	0.01	0.36	1.24	0.03	0.03	0.32	0.41	0.15	0.06
Claude-Sonnet-4.5	Accuracy	0%	6.67%	0%	33.33%	8.33%	35.00%	66.67%	40.00%	38.33%	76.67%
	Cost (\$)	0.91	0.03	0.05	2.42	0.16	0.10	0.80	1.45	0.35	0.24
Stratified Accuracy under GPT-5	Easy (n=17)	0%	23.53%	0%	35.29%	35.29%	70.59%	52.94%	5.88%	64.71%	100.00%
	Medium (n=33)	0%	12.12%	0%	30.30%	33.33%	54.55%	42.42%	18.18%	51.52%	81.82%
	Hard (n=10)	0%	0%	0%	20.00%	10.00%	30.00%	30.00%	10.00%	30.00%	80.00%

Future Directions

What We Now Know



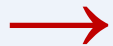
The operator zoo is aligned

Five intents × four granularities × implementation variants — one taxonomy. (Q1)



Best practices are measured

A tested default for every intent-by-operand cell — with quality and cost. (Q2)



Systems remain early-stage

Planning, coordination, optimization: we have answers — not yet solutions. (Q3–Q4)

The field is organized — and unfinished. Here is exactly where.

Three Walls — Measured, Not Guessed

THE PLANNING WALL

<15%

accuracy of automatic
LLM planners

*0 / 10 hard queries solved
by any planning system*

THE HALLUCINATION WALL

100%

of best-practice failures
are hallucination

*the last failure standing —
even when everything else works*

THE CONTEXT WALL

~2k rows

where LLM-ALL collapses —
far below the 400k-token
window

*and up to 90× cost spread
across implementation choices*

Every wall is measured — and every wall is a research agenda. One direction for each.

Direction 1 - Robust LLM Planning

BREAKS THE PLANNING WALL

Generate – verify – repair harnesses

Trust no single model by default: candidate plans are checked, repaired, or regenerated.

Learning to plan: RL and prompt optimization

Treat plan generation as a trainable skill, not a fixed prompt.

Plan equivalence checking before execution

Verify that rewrites and placements preserve semantics — symbolically, not by trust.

A wrong plan cannot be optimized into a right one.

Direction 2 · Verification & Semantic Assurance

BREAKS THE HALLUCINATION WALL

Verification without an oracle

No compiler, no unit tests — data agents still need something that says “this result is right.”

Traceable, auditable results

Every output carries its evidence: which rows, which calls, which model, which prompt.

Containment at the operator boundary

Typed outputs, constrained decoding, cross-checks — catch hallucination where it enters.

When hallucination is the last failure standing, verification is the next wall to build.

Direction 3 · Cost-Aware Execution

BREAKS THE CONTEXT WALL

A quality–cost curve per model — measured

Every base model has its own effective-context frontier; engines should learn it, not assume it.

Smart batching between the two extremes

Neither one giant prompt nor one call per tuple — batches the model can actually hold.

Joint optimization of SQL and semantic operators

One planner, two cost regimes — relational work is nearly free; model calls dominate.

The sweet spot exists — around 50–100 tuples per batch in our runs. Engines should find it automatically.

The Long Term: LLM-Centric Databases

Not LLMs as plugins — databases rebuilt around operators that reason.

Multi-modal unified storage

tables, text, images, video — one substrate

Semantic-aware indexing

beyond traditional indexes and vanilla RAG

Adaptive context management

prompts constructed dynamically, per query

Holistic optimization

storage · indexing · planning · model invocation —
one optimizer

At the core: LROs — the abstraction layer that embeds LLM capability into query processing.

Fifty years ago, a closed algebra over exact predicates built an industry.

The next fifty may be built on operators that reason.

Thank You!